Mini-projet — Système de supervision d'un environnement informatique

Groupes de 2 et 3 étudiants

Objectif

Réaliser une application web de supervision capable de simuler plusieurs machines et d'afficher leurs mesures en temps réel. L'application devra permettre :

- la génération de données dynamiques par un backend Python ;
- la récupération de ces données via une API Flask ;
- l'affichage des valeurs dans une interface web ;
- la visualisation de l'évolution d'une mesure à l'aide d'un graphique.

L'ensemble devra fonctionner selon la chaîne suivante :

Python → JSON → JavaScript (`fetch`) → Interface web → Graphique.

1. **Python (backend)** → génère des données (CPU, température…)
2. **API Flask** → rend ces données accessibles via une URL (`/data`)
3. **JSON** → format de transmission des données
4. **JavaScript (`fetch`)** → récupère les données depuis l'API
5. **Interface Web (HTML/CSS)** → affiche les données
6. **Graphique (Chart.js)** → montre l'évolution dans le temps

> C'est une **chaîne complète**, et chaque partie doit fonctionner avec les autres.

Préparation de l'environnement de travail

Utilisation d'un environnement virtuel

- Pour installer des bibliothèques Python localement, sans modifier l'installation système.
- Pour l'**isolation des projets** : Chaque projet a ses propres dépendances, évitant les conflits entre versions.
- Pour la **portabilité** : Facile à partager ou à recréer sur une autre machine.

Étapes pour créer et activer un environnement virtuel

1. Créer un dossier pour le projet

- Ouvrez l'explorateur de fichiers et créez un dossier nommé `projet_supervision` sur votre bureau ou dans vos documents.

2. Ouvrir un terminal (ou invite de commandes)

- **VS Code** : Ouvrez le terminal
- **Windows** : `Win + R` → tapez `cmd` → Entrée.
- **Mac/Linux** : Ouvrez le terminal depuis le dossier utilisateur.

3. Se placer dans le dossier du projet

- Dans le terminal, tapez :

  ```
  cd Chemin/vers/votre/dossier/projet_supervision
  ```

  (Remplacez `Chemin/vers/votre/dossier` par le chemin réel, par exemple `cd Desktop/projet_supervision` sur Windows.)

4. Créer l'environnement virtuel

- Tapez la commande suivante :

  ```
  python -m venv venv
  ```

  - Cela crée un dossier `venv` contenant une copie isolée de Python et de pip.

5. Activer l'environnement virtuel

- **Windows** :

  ```
  venv\Scripts\activate
  ```

  → Votre invite de commande doit maintenant commencer par `(venv)`.

6. Installer les bibliothèques nécessaires

- Avec l'environnement activé, installez Flask et Flask-CORS :

  ```
  pip install flask flask-cors
  ```

  → Les bibliothèques sont installées **uniquement** dans votre environnement virtuel.

7. Vérifier l'installation

- Tapez :

  ```
  pip list
  ```

→ Vous devriez voir `flask` et `flask-cors` dans la liste.

8. Désactiver l'environnement virtuel (quand vous avez fini)

- Tapez :

```
deactivate
```

Utilisation au quotidien

- **Pour travailler sur le projet** :
  1. Ouvrez un terminal.
  2. Placez-vous dans le dossier du projet (`cd Chemin/vers/projet_supervision`).
  3. Activez l'environnement (`venv\Scripts\activate` ou `source venv/bin/activate`).
  4. Lancez votre API (`python backend/app.py`).
- **Pour installer une nouvelle bibliothèque** :
  - Activez l'environnement, puis utilisez `pip install nom_de_la_bibliothèque`.

Partage du projet avec d'autres étudiants du groupe

- **Ne partagez pas le dossier `venv`** (il est spécifique à votre machine).
- **Partagez uniquement** :
  - Le code source (`backend/`, `frontend/`).
  - Un fichier `requirements.txt` (voir ci-dessous).

Créer un fichier `requirements.txt`

- Avec l'environnement activé, tapez :

```
pip freeze > requirements.txt
```

- Ce fichier liste toutes les bibliothèques installées. Un autre étudiant du groupe pourra les installer avec :

```
pip install -r requirements.txt
```

Résolution des problèmes courants

Problème	Solution

Problème	Solution
`python -m venv` ne fonctionne pas	Essayez `python3 -m venv venv`.
L'activation échoue sur Windows	Vérifiez que vous êtes dans le bon dossier et que le chemin ne contient pas d'espaces.
`pip` n'est pas reconnu	Activez d'abord l'environnement virtuel.
Erreur de permission	Utilisez toujours l'environnement virtuel, pas l'installation système.

Bonnes pratiques

- **Toujours activer l'environnement** avant de lancer votre API ou d'installer des bibliothèques.
- **Ne modifiez pas le dossier `venv`** manuellement.
- **Partagez `requirements.txt`** pour que vos coéquipiers puissent recréer l'environnement facilement.

Une fois cette étape terminée, vous pouvez passer à la création de l'API Flask (Rôle 1) ou du frontend (Rôle 2).

Organisation du projet

Le projet est réalisé en groupe de **2 ou 3 étudiants**.
La répartition des rôles est obligatoire, mais chaque membre doit comprendre l'ensemble de l'application.

Fonctionnement attendu

L'application doit :

1. simuler plusieurs machines ;
2. produire des données dynamiques ;
3. exposer ces données via une route API ;
4. afficher les informations dans une page web ;
5. mettre à jour automatiquement l'affichage toutes les 5 secondes ;
6. afficher un graphique représentant l'évolution d'une donnée dans le temps.

Données

L'API devra retourner, pour **au moins 3 machines**, les informations suivantes :

- nom de la machine ;
- température, comprise entre 20 et 80 °C ;
- CPU, compris entre 0 et 100% ;
- timestamp correspondant à l'heure actuelle.

Exemple de réponse JSON :

```
[
  {
    "nom": "Serveur-1",
    "temperature": 45,
    "cpu": 32,
    "timestamp": "14:32:10"
  },
  {
    "nom": "Serveur-2",
    "temperature": 51,
    "cpu": 67,
    "timestamp": "14:32:10"
  },
  {
    "nom": "Serveur-3",
    "temperature": 38,
    "cpu": 21,
    "timestamp": "14:32:10"
  }
]
```

Les valeurs doivent changer à chaque appel de l'API.

Architecture attendue

Le projet doit suivre cette organisation :

```
projet_supervision/
│
├─ backend/
│   └─ app.py
│
├─ frontend/
│   ├─ index.html
│   ├─ style.css
│   └─ script.js
│
├─ data/
│   └─ data.json (optionnel)
│
└─ README.txt
```

Flask utilise classiquement un dossier `templates` pour les pages HTML et un dossier `static` pour les fichiers CSS/JavaScript, ce qui peut être proposé comme organisation alternative si vous souhaitez une structure plus standardisée.

Mission A — Backend Python (Flask)

Objectif

Créer une API Flask capable de fournir des données dynamiques de supervision. Flask documente précisément l'usage de routes JSON et d'échanges avec JavaScript via `fetch`.

À réaliser

- créer une application Flask ;
- créer une route `/data` ;
- générer des données aléatoires ou simulées à chaque requête ;
- renvoyer les données au format JSON.

Contraintes

- l'API doit fonctionner seule ;
- elle doit pouvoir être testée dans un navigateur ;
- les données doivent être renouvelées à chaque appel.

Mission B — Frontend (Interface Web)

Objectif

Créer une interface web lisible et dynamique permettant d'afficher les mesures des machines. L'affichage doit être mis à jour sans rechargement de page grâce à `fetch`.

À réaliser

- créer une page HTML ;
- afficher les machines sous forme de tableau ou de cartes ;
- afficher au minimum le CPU et la température ;
- utiliser `fetch` pour récupérer les données ;
- actualiser l'affichage automatiquement toutes les 5 secondes.

Contraintes

- aucun rechargement de page ;
- l'interface doit rester lisible ;
- le code JavaScript doit être clair et compréhensible.

Exemple JS :

```
setInterval(() => {
  fetch("http://localhost:5000/data")
    .then(response => response.json())
    .then(data => {
      // afficher les données
    });
}, 5000);
```

> C'est ici que vous faites le lien entre backend et affichage.

> Vérifiez que le backend est bien lancé sur le port 5000.Et éventuellement :
>
> Sur certains systèmes : `127.0.0.1` au lieu de `localhost`.

Mission C — Graphique

Objectif

Afficher l'évolution d'une donnée dans le temps à l'aide de Chart.js ou d'une librairie équivalente. Chart.js permet de modifier les tableaux de données du graphique puis d'appeler `update()` pour rafraîchir l'affichage.

À réaliser

- afficher l'évolution du CPU ou de la température ;
- conserver au moins 10 valeurs ;
- mettre à jour le graphique automatiquement.

Niveau attendu

- **groupe de 2** : graphique simple sur une mesure ;
- **groupe de 3** : graphique plus avancé, éventuellement avec plusieurs machines ou plusieurs courbes.

Répartition des rôles

Consignes générales pour tous les rôles

- **Communiquer régulièrement** entre vous pour vérifier la compatibilité des parties (ex : format du JSON, URL de l'API).
- **Tester chaque étape** avant de passer à la suivante.
- **Utiliser Git** (ou un partage de fichiers) pour collaborer efficacement.

Groupe de 3

- **Étudiant 1 — Backend / API**
  - créer l'API Flask ;
  - générer les données ;
  - structurer le JSON.
- **Étudiant 2 — Frontend / Interface**
  - créer la page web ;
  - afficher les données ;
  - utiliser `fetch` ;
  - gérer la mise à jour automatique.
- **Étudiant 3 — Graphique / Qualité**
  - intégrer Chart.js ;
  - gérer l'historique ;

    - effectuer les tests ;
    - corriger les bugs.

Groupe de 2

- **Étudiant 1 — Backend et données**
    - API Flask ;
    - génération des données ;
    - structure JSON.
- **Étudiant 2 — Frontend et visualisation**
    - interface web ;
    - `fetch` ;
    - affichage dynamique ;
    - graphique.

Le travail de tests et de correction est commun aux deux étudiants.

Tests obligatoires

Avant la remise, vous devez vérifier que :

- [ ] l'API fonctionne ;
- [ ] le JSON est valide ;
- [ ] `fetch` récupère correctement les données ;
- [ ] l'affichage est correct ;
- [ ] la mise à jour automatique fonctionne ;
- [ ] le graphique se met à jour correctement.

Livrables

1. Code source

Le code doit être :

- complet ;
- organisé ;
- lisible ;
- fonctionnel.

2. Rapport

Le rapport doit faire environ **3 à 4 pages** et contenir :

- une présentation du projet ;
- une description de l'architecture ;
- l'explication technique du fonctionnement ;
- les difficultés rencontrées ;
- la répartition du travail dans le groupe.

3. Notice d'utilisation

La notice doit préciser :

- les étapes d'installation ;
- le lancement du backend ;
- l'ouverture du frontend ;
- l'URL à utiliser pour accéder à l'application.

Critères d'évaluation

Technique — 12 points

Critère	Points
API fonctionnelle	4
Frontend dynamique	4
Graphique	2
Fonctionnement global	2

Documentation — 4 points

Critère	Points
Rapport	2
Notice d'utilisation	2

Travail d'équipe — 4 points

Critère	Points
Répartition du travail	2
Compréhension individuelle	2

Bonus facultatifs

- code couleur selon les seuils ;
- plusieurs courbes ;
- interface plus soignée ;
- sauvegarde des données ;
- route supplémentaire `/data/<machine>`.

Consignes importantes

- commencer par une version simple ;
- tester régulièrement chaque partie ;
- vérifier le fonctionnement de bout en bout ;

- s’assurer que chaque membre comprend l’ensemble du projet.